UPDATE : 13/11/20

UPDATE : 15/11/20

Machine utilisée : Linux

Implémentation et étude des modules dans l'ordre de mon planning :

1) Balises ✔

Module permettant le traitement des balises choisies par l'utilisateur à l'usage d'un programme.

Développement d'un header « dh_bal.h » qui dépend des balises qui vont être requis pour l'utilisateur.
Point sur les balises requis :

- -i : Fichier source.
- -o : Fichier destination.
- -k : Clé pour le chiffrement ou longueur de la clé.
- -d : Dictionnaire.
- -m : Méthode.
- -h : L'aide pour l'utilisation du programme.

Traitement des balises par des fonctions dans un fichier « dh_bal.c » :

- Exemple : « est_balide_i » qui retourne un booléen en fonction de si l'élément à l'indice donné correspond à la balise -i.

Le header « dh_bal.c » sera utilisée par chaque module pour faciliter le traitement des balises.

Remarque : association du « dh_bal.h » avec le « dh_bal.c ».

2) XOR ✔

Module de chiffrement avec l'opérateur xor caractère par caractère qui n'a aucune dépendance vis-à-vis des autres modules.

- Développement d'un fichier c comprenant la fonction « xor » qui utilisera des fonctions qui lui seront complémentaires :
  - Fonction vérification de l'allocation.
  - Fonction lecture du nombre d'octets du contenu du fichier source à chiffrer.
    - Remarque : Nécessite une boucle dépendante du nombre d'octets contenu dans le fichier source afin de tous les traiter.
  - Fonction de test qui sera appelée à chaque utilisation de la fonction XOR qui permet de vérifier le bon fonctionnement du chiffrage et du déchiffrage.
    - Remarque : Chiffrage et déchiffrage d'un fichier source avec un assert pour vérifier les contenus.

  Remarque : Utilisation d'une boucle pour la fonction « xor » dans le but de traiter tous les octets du fichier source.

- Développement du main « dh_crypt.c » pour traiter la demande d'usage du programme de l'utilisateur en fonction des balises utilisées avec le header « dh_bal.h » :

- -i : Fichier source.
- -o : Fichier destination.
- -k : Clé pour le chiffrement.
- -m : Méthode XOR ou CBC.
- -h : Affichage aide pour l'utilisation du programme.

3) DH ✔

Module génération d'un nombre premier Sophie-Germain et de son générateur qui n'a aucune dépendance vis-à-vis des autres modules.

- Réutilisation de toutes les fonctions données dans le fichier « Code C pour dh_gen » et développement de la fonction « xchange_shared_key » qui a pour but de simuler l'échange de clefs de Diffie-Hellman par réseau. Toutes ces fonctions réunies dans un seul fichier c.
  Pour se faire :
  - Réutilisation du header donné « dh.h ».
  - Appel aux fonctions « random_long » et « puissance_mod_n » sur les paramètres « premier » et « generateur » de la fonction « xchange_shared_key ».
    - Remarque : Stockage des résultats dans des variables.
- Développement du main « dh_genkey.c » pour traiter la demande d'usage de l'utilisateur en fonction des balises utilisées avec le header « dh_bal.h » :
  - Aucune balise : Affichage du protocole.
  - -o : Fichier destination.
  - -h : Affichage aide pour l'utilisation du programme.

4) C1

Module crack d'une clé par émondage des caractères possible pour la clef qui n'a aucune dépendance vis-à-vis des autres modules.

- Développement d'un fichier c « dh_crack_c1.c » comprenant la fonction « c1 » avec utilisation de fonctions qui lui sont complémentaires :
  - Fonction vérification d'allocation.
  - Fonction vérification si l'octet donné est admissible avec le caractère de la clé donné.
    - Remarque : Fonction booléenne.
  - Fonction lecture des octets contenu dans le fichier source.
    - Remarque : Utilisation d'un tableau dynamique pour stocker les octets contenus dans le fichier source.
  - Fonction qui permet d'afficher le produit cartésien des clefs.
    - Remarque : Potentiellement plusieurs boucles imbriquées.
  - Réutilisation du header donné « dh_crack.h »

  Remarques :

            - Forte probabilité d'une méthode de traitement par l'utilisation d'une matrice pour les listes des caractères admissibles pour chaque clef.
                - Possibilité d'une structure comprenant un tableau dynamique à deux dimensions, et d'une variable stockant la longueur de la clé.
            - Utilisation d'une boucle dépendante de la longueur de la clé pour bien traiter toutes les listes de clé.
                - Implémentée d'une boucle imbriquée pour traiter les octets k+1, 2k+1 etc. (k étant la longueur de la clé)
- Développement du main « dh_crack.c » pour traiter la demande d'usage de l'utilisateur en fonction des balises utilisées avec le header « dh_bal.h » :
    - -i : Fichier source.
    - -m : Méthode c1 ou all.
    - -k : Longueur de la clé.
    - -d : Dictionnaire
    - -h : Affichage aide pour l'utilisation du programme.

5) <u>CBC</u>

Module de chiffrement par blocs d'octets avec l'opérateur xor. Reprise de la fonction « xor » précédente mais de manière plus élaborée.

- Développement d'un fichier c comprenant la fonction « cbc » utilisant des fonctions qui lui sont complémentaires :
    - Fonction vérification d'allocation.
    - Réutilisation du corps de la fonction « xor » avec les changements adéquates : chiffrage sur des blocs d'octets et avec un vecteur d'initialisation qui est une chaîne de caractères.
        - <u>Remarque :</u> Utilisation potentiel de tableaux dynamiques.

  <u>Remarque :</u> Utilisation potentielle d'une boucle dépendante du nombre blocs d'octets pour bien tous les traiter.
- Développement d'une fonction « dechiffrage_cbc » qui permettra de déchiffrer un texte contenu dans un fichier source qui a été au préalable chiffré avec la fonction « cbc ».
- Les fonctions « cbc » et « dechiffrage_cbc » seront associées au main « dh_crypt.c » avec la fonction « xor » déjà implémentée.

  <u>Remarque :</u> Les fonctions « cbc » et « dechiffrage_cbc » seront dans le même fichier c et la fontion « xor » dans un autre fichier c.

6) <u>C2</u>

Module crack d'une clé par statistiques des lettres associées au fait qu'avec un chiffrement type xor caractère par caractère, une même lettre risque d'être chiffrée par le même nombre. Dépendance notable de C2 avec C1 car on a un traitement du cryptogramme avec chaque clef candidate obtenue par la méthode C1.

- Développement d'un fichier c « dh_crack_c2.c » comprenant la fonction « c2 » avec utilisation de fonctions qui lui sont complémentaires :
    - Fonction de vérification de l'allocation.
    - Réutilisation du corps de la fonction « c1 » du fichier « dh_crack_c1.c ».
        - Possibilité d'utilisation d'un tableau dynamique comprenant la statistique de chaque lettre.
        - Possibilité d'utilisation d'une boucle dépendante du nombre d'octets situés dans le fichier source pour bien traiter tout le contenu.

  Remarque : Nécessité de l'utilisation du fichier Excel « Fichier de fréquences de lettres » donné.
- La fonction « c2 » sera associée au main « dh_crack.c » avec la fonction « c1 » déjà implémentée.

Nombre maximum de points potentiels : 17 points.